Categorical Model

General Principles

To model the relationship between a outcome variable in which each observation is a single choice from a set of more than two categories and one or more independent variables, we can use a *Categorical* model.

Considerations

Caution

- We have the same considerations as for Regression for continuous variable.
- One way to interpret a *Categorical* model is to consider that we need to build $K-1$ linear models, where K is the number of categories. Once we get the linear prediction for each category, we can convert these predictions to probabilities by building a simplex . To do this, we convert the regression outputs using the softmax function (see the "jax.nn.softmax" line in the code).
- The intercept α captures the difference in the log-odds of the outcome categories; thus, different categories need different intercepts.
- On the other hand, as we assume that the effect of each predictor on the outcome is consistent across all categories, the regression coefficients β are shared across categories.
- The relationship between the predictor variables and the log-odds of each category is modeled linearly, allowing us to interpret the effect of each predictor on the log-odds of each category.

Example

Below is an example code snippet demonstrating a Bayesian Categorical model using the BayesForge (BF) package. This example is based on McElreath (2018). The covariate (income) varies across individuals and careers: with the same incomes for everyone, the slope b could not be distinguished from the intercepts.

Python

```python
from BayesForge import bf
import jax.numpy as jnp
import jax
# Setup device ------------------------------------------------
m = bf('cpu')

# Simulate data -----------------------------------------------
# 300 people each choose among 3 careers; income[i, k] is the income
# person i would earn in career k. Income must vary across people,
# otherwise b cannot be told apart from the intercepts a.
N = 300
income = m.dist.uniform(1, 10, shape=(N, 3), sample=True, seed=1)
a_true, b_true = jnp.array([0.5, -0.2]), 0.4   # career 3 is the pivot
scores = jnp.stack([a_true[0] + b_true * income[:, 0],
                    a_true[1] + b_true * income[:, 1],
                    b_true * income[:, 2]], axis=1)
career = m.dist.categorical(probs=jax.nn.softmax(scores, axis=1), sample=True, seed=0)
m.data_on_model = dict(career=career, inc0=income[:, 0], inc1=income[:, 1], inc2=income[:, 2

# Define model ------------------------------------------------
def model(career, inc0, inc1, inc2):
    a = m.dist.normal(0, 1, shape=(2,), name='a')
    b = m.dist.half_normal(1.0, name='b')
    s_1 = a[0] + b * inc0
    s_2 = a[1] + b * inc1
    s_3 = b * inc2   # pivot: no intercept
    p = jax.nn.softmax(jnp.stack([s_1, s_2, s_3], axis=1), axis=1)
    m.dist.categorical(probs=p, obs=career)

# Run sampler -------------------------------------------------
m.fit(model, progress_bar=False)  # Optimize model parameters through MCMC sampling
```

```
# Summary ----------------------------------------------
m.summary() # Get posterior distributions
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
a[0]	0.63	0.16	0.38	0.88	2871.99	2795.00	1.0	0.0	0.0
a[1]	-0.19	0.17	-0.46	0.08	2693.88	2636.09	1.0	0.0	0.0
b	0.39	0.04	0.32	0.45	2925.93	2476.02	1.0	0.0	0.0

R

```
library(BayesForge)
jax = reticulate::import('jax')
jnp = reticulate::import('jax.numpy')

# setup platform----------------------------------------------
m=importBF(platform='cpu')

# Simulate data ----------------------------------------------
set.seed(1)
N <- 300L
income <- matrix(runif(N * 3, 1, 10), N, 3)
scores <- cbind(0.5 + 0.4 * income[, 1], -0.2 + 0.4 * income[, 2], 0.4 * income[, 3])
p_true <- exp(scores) / rowSums(exp(scores))
career <- apply(p_true, 1, function(p) sample(0:2, 1, prob = p))
m$data_on_model <- list(career = jnp$array(as.integer(career)),
                        inc0 = jnp$array(income[, 1]), inc1 = jnp$array(income[, 2]),
                        inc2 = jnp$array(income[, 3]))

# Define model ----------------------------------------------
model <- function(career, inc0, inc1, inc2){
  a = m$dist$normal(0, 1, shape = reticulate::tuple(2L), name = 'a')
  b = m$dist$half_normal(1.0, name = 'b')
  s_1 = a[0] + b * inc0
```

```
  s_2 = a[1] + b * inc1
  s_3 = b * inc2   # pivot: no intercept
  p = jax$nn$softmax(jnp$stack(list(s_1, s_2, s_3), axis = 1L), axis = 1L)
  m$dist$categorical(probs = p, obs = career, name = "career")
}

# Run MCMC -----------------------------------------------
m$fit(model) # Optimize model parameters through MCMC sampling

# Summary ------------------------------------------------
m$summary() # Get posterior distribution
```

Julia

```
using PyBayesForge

# Setup device-------------------------------------------------
m = importBF(platform="cpu")

using Random
# Simulate data ------------------------------------------------
rng = MersenneTwister(1)
N = 300
income = 1 .+ 9 .* rand(rng, N, 3)
scores = hcat(0.5 .+ 0.4 .* income[:, 1], -0.2 .+ 0.4 .* income[:, 2], 0.4 .* income[:, 3])
p_true = exp.(scores) ./ sum(exp.(scores), dims=2)
career = [findfirst(cumsum(p_true[i, :]) .>= rand(rng)) - 1 for i in 1:N]
m.data_on_model = Dict("career" => jnp.array(career), "inc0" => jnp.array(income[:, 1]),
                       "inc1" => jnp.array(income[:, 2]), "inc2" => jnp.array(income[:, 3]))

# Define model -------------------------------------------------
@BF function model(career, inc0, inc1, inc2)
    a = m.dist.normal(0, 1, shape=(2,), name = "a")
    b = m.dist.half_normal(1.0, name = "b")
    s_1 = a[0] + b * inc0
    s_2 = a[1] + b * inc1
    s_3 = b * inc2   # pivot: no intercept
    p = jax.nn.softmax(jnp.stack([s_1, s_2, s_3], axis=1), axis=1)
    m.dist.categorical(probs=p, obs=career)
end
```

```
# Run mcmc -----------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling

# Summary ------------------------------------------------
m.summary() # Get posterior distributions
```

Mathematical Details

We can model a *Categorical* model using a *Categoricaldistribution*. The multinomial distribution models the counts of outcomes falling into different categories. For an outcome variable with categories, the multinomial likelihood function is:

$$Y_i \sim \text{Categorical}(\theta_i)\theta_i = \text{Softmax}(\phi_i)\phi_{[i,1]} = \alpha_1 + \beta_1 X_i \phi_{[i,2]} = \alpha_2 + \beta_2 X_i ... \phi_{[i,k]} = 0\alpha_k \sim \text{Normal}(0,1)\beta_k \sim \text{Norma}$$

Where:

- Y_i is the dependent categorical variable for observation i indicating the category of the observation.
- θ_i is a vector unique to each observation, i, which gives the probability of observing i in category k.
- ϕ_i give the linear model for each of the k categories. Note that we use the softmax function to ensure that that the probabilities θ_i form a simplex .
- Each element of ϕ_i is obtained by applying a linear regression model with its own respective intercept α_k and slope coefficient β_k. To ensure the model is identifiable, one category, K, is arbitrarily chosen as a reference or baseline category. The linear predictor for this reference category is set to zero. The coefficients for the other categories then represent the change in the log-odds of being in that category versus the reference category.

Model validation tests

- Comparison with Stan: `8.Multinomial.py` fits the same model with BayesForge and Stan and compares the posterior distributions (KL divergence).
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan*. Chapman; Hall/CRC.